



Centrifuge Workflows Audit Report

Prepared by [Burra Security](#)

Version 1.0

Researchers

fuzious

Taridoku

typicalHuman

patitonar

Kiki

June 19, 2026

Contents

1	About Burra Security	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	2
7	Findings	5
7.1	High Risk	5
7.1.1	BalanceSheet.withdraw() Is Void - Morpho Loop Deposit Captures Garbage Equity Amount	5
7.1.2	Strategist Can Inflate NAV via Stale Aave aToken Accounting	5
7.1.3	Share oracle workflow stores raw asset units as D18 price	6
7.1.4	ERC-7540 receipt token collision merges deposit and redeem accounting	6
7.1.5	cctp_claim Strategist-Controlled Rawcalldata Enables Arbitrary MessageTransmitter Invocation	7
7.1.6	ERC-7540 claim-redeem can permanently lock later fulfilled redemptions by over-burning accounting tokens	7
7.1.7	Oracle update target uses wrong bytes32 address encoding	8
7.2	Medium Risk	9
7.2.1	cfg.scriptHelpers Falsy Check Does Not Catch Zero Address String	9
7.2.2	Share oracle workflow assumes underlying asset is 1:1 with pool currency	9
7.2.3	Inconsistent bytes32 Mint Recipient Encoding Across CCTP Send Templates May Lead to Unrecoverable Minting	10
7.2.4	CCTP automatic send always reverts	10
7.2.5	Shared \$sclD Circuit-Breaker Key Mixes Egress Flow Limits Across Assets and Units	11
7.3	Low Risk	12
7.3.1	Sync VaultKind Silently Falls Through to Async Templates	12
7.3.2	CCTP maxFee Is Hardcoded Across All Routes	12
7.3.3	chainName Not Sanitized - Hyphens in Workflow IDs on Testnet Chains	13
7.3.4	scriptHelpers Passed to Balance-Sheet Workflows Whose Templates Don't Declare It	13
7.3.5	Wrong Parameter Labels on requestDeposit and requestRedeem	14
7.3.6	Morpho loop callback can permanently strand excess repayment in FlashLoanHelper	14
7.3.7	erc7540_claimRedeem Missing Slippage Guard	15
7.3.8	Aave Templates Operate on Full aToken Balance Instead of the Exact Amount	15
7.3.9	BalanceSheet Revoke Can Burn Any Holder's Shares Approved to OnchainPM	16
7.3.10	Generator workflow IDs collide and silently drop vaults	16
7.3.11	Build and publish skip validate	17
7.4	Informational	18
7.4.1	ERC-7540 workflows omit required queued-asset submissions	18
7.4.2	Mainnet address pins set ScriptHelpers to zero	18
7.4.3	Guarded workflows run unguarded with zero pins	19
7.4.4	Rounding labels omit Zero	19
7.4.5	Generated workflows reference missing JSON Schema path	20
7.4.6	ERC-7540 request workflows can freeze PoolEscrow availability by double-reserving requested tokens	20

1 About Burra Security

Burra Security offers security auditing and advisory services with a special focus on cross-chain and interoperability protocols and their integrations. Learn about us at burrasec.com.

2 Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any vulnerabilities. Subsequent security reviews, bug bounty programs, and on-chain monitoring are recommended.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Centrifuge is an open, decentralized protocol for onchain real-world asset management, enabling permissionless deployment of tokenization products — from permissioned funds to onchain loans — across multiple EVM networks.

The audited Workflows repository defines declarative DeFi workflows for the Weiroll executor. Reusable JSON templates describe action sequences with `$variable` placeholders, and TypeScript generators resolve them into concrete workflow instances with pinned contract addresses. These workflows orchestrate operations against external protocols — Aave V3, Morpho, and Circle CCTP — as well as Centrifuge's own pool contracts.

The Centrifuge flows target ERC-7540 async vaults, which decouple deposit and redemption requests from settlement, alongside ERC-4626-compatible deposit and redeem actions. A shared workspace provides reusable primitives consumed by the mainnet deployment, including ERC-20/4626/7540 templates, math and encoding helpers, and oracle-price and balance-sheet templates.

5 Audit Scope

The following smart contracts were in the scope of the audit:

```
workspaces/  
├── mainnet/  
│   └── *  
├── shared/  
│   └── *  
└── *
```

6 Executive Summary

Over the course of 5 days, the Burra Security team conducted an audit on the [Centrifuge Workflows](#) smart contracts provided by [Centrifuge](#). In this period, a total of 29 issues were found.

Summary

Project Name	Centrifuge Workflows
Repository	workflows
Commit	1df4fda6df69...
Audit Timeline	22nd May - 28th May
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	7
Medium Risk	5
Low Risk	11
Informational	6
Gas Optimizations	0
Total Issues	29

Summary of Findings

[H-1] <code>BalanceSheet.withdraw()</code> Is Void - Morpho Loop Deposit Captures Garbage Equity Amount	Resolved
[H-2] Strategist Can Inflate NAV via Stale Aave <code>aToken</code> Accounting	Resolved
[H-3] Share oracle workflow stores raw asset units as D18 price	Resolved
[H-4] ERC-7540 receipt token collision merges deposit and redeem accounting	Resolved
[H-5] <code>cctp_claim</code> Strategist-Controlled Rawcalldata Enables Arbitrary MessageTransmitter Invocation	Resolved
[H-6] ERC-7540 claim-redeem can permanently lock later fulfilled redemptions by over-burning accounting tokens	Resolved
[H-7] Oracle update target uses wrong bytes32 address encoding	Resolved
[M-1] <code>cfg.scriptHelpers</code> Falsy Check Does Not Catch Zero Address String	Acknowledged
[M-2] Share oracle workflow assumes underlying asset is 1:1 with pool currency	Resolved
[M-3] Inconsistent bytes32 Mint Recipient Encoding Across CCTP Send Templates May Lead to Unrecoverable Minting	Resolved
[M-4] CCTP automatic send always reverts	Resolved
[M-5] Shared <code>\$scl</code> Circuit-Breaker Key Mixes Egress Flow Limits Across Assets and Units	Resolved
[L-01] <code>Sync VaultKind</code> Silently Falls Through to Async Templates	Resolved
[L-02] CCTP <code>maxFee</code> Is Hardcoded Across All Routes	Resolved

[L-03] chainName Not Sanitized - Hyphens in Workflow IDs on Testnet Chains	Resolved
[L-04] scriptHelpers Passed to Balance-Sheet Workflows Whose Templates Don't Declare It	Resolved
[L-05] Wrong Parameter Labels on requestDeposit and requestRedeem	Resolved
[L-06] Morpho loop callback can permanently strand excess repayment in FlashLoanHelper	Resolved
[L-07] erc7540_claimRedeem Missing Slippage Guard	Resolved
[L-08] Aave Templates Operate on Full aToken Balance Instead of the Exact Amount	Resolved
[L-09] BalanceSheet Revoke Can Burn Any Holder's Shares Approved to On-chainPM	Acknowledged
[L-10] Generator workflow IDs collide and silently drop vaults	Resolved
[L-11] Build and publish skip validate	Resolved
[I-1] ERC-7540 workflows omit required queued-asset submissions	Resolved
[I-2] Mainnet address pins set ScriptHelpers to zero	Resolved
[I-3] Guarded workflows run unguarded with zero pins	Resolved
[I-4] Rounding labels omit Zero	Resolved
[I-5] Generated workflows reference missing JSON Schema path	Resolved
[I-6] ERC-7540 request workflows can freeze PoolEscrow availability by double-reserving requested tokens	Acknowledged

7 Findings

7.1 High Risk

7.1.1 `BalanceSheet.withdraw()` Is Void - Morpho Loop Deposit Captures Garbage Equity Amount

Target: [workspaces/mainnet/templates/morpho-loop-deposit.json](#)

Severity:

- Impact: High
- Likelihood: High

Description: The first action in the `morpho_loop_deposit` template calls `BalanceSheet.withdraw(uint64,byte s16,address,uint256,address,uint128)` and declares `"returns": "$equityAmount"`. However, this function is void - it transfers tokens to the receiver and returns nothing. Every other template that calls the same selector (e.g. `c4626_deposit`, `erc7540_requestDeposit`, `aave_supply`, etc.) correctly omits `returns` on this action. In `weiroll`, declaring `returns` on a void call writes empty return data to the state slot, so `$equityAmount` resolves to 0. The next action computes `$flashAmount = mul($equityAmount=0, $flashMultiplier) = 0`, meaning the flash loan is requested for zero tokens. The leverage loop deposits only the equity without any borrowed multiplier - the entire leveraging mechanism silently does nothing.

Recommended Mitigation: Remove `"returns": "$equityAmount"` from the `BalanceSheet.withdraw` action and replace downstream references to `$equityAmount` with the runtime variable `$amount`, which already holds the equity value passed in by the strategist. The `mul` action becomes `mul($amount, $flashMultiplier)`.

Client: <https://github.com/centrifuge/workflows/pull/66/changes/4d35b2ccbeabf2e6caafe2825765ab13292b4e58>

BurraSec: The solution correctly resolves the no-output issue by replacing it with the `$amount` input parameter.

7.1.2 Strategist Can Inflate NAV via Stale Aave aToken Accounting

Target: [aave-account.json](#), [balancesheet-note-deposit.json](#), [balancesheet-note-withdraw.json](#), [BalanceSheet.sol](#), [PoolEscrow.sol](#).

Severity: High

- Impact: High
- Likelihood: High

Description: Aave aTokens can rebase while being held in `PoolEscrow`, so the live ERC20 aToken balance can diverge from `PoolEscrow.holding.total`. The `aave-account` workflow detects this delta, but its yield path calls `noteDeposit` with `updateEscrow = false`, which will `skip` escrow accounting updates. Note that `aave_supply`, through `bs_approve_and_deposit`, deposits into the `PoolEscrow` via `BalanceSheet.deposit`, which `does` call `noteDeposit(..., updateEscrow = true)`. This means positive rebase yield can remain physically held by the `PoolEscrow` while being unavailable through normal `BalanceSheet.withdraw` flows, necessitating privileged recovery. More importantly, since `availableBalanceOf` stays stale, a strategist can use `aave_account` to resubmit the same `$yield` over and over. Each rerun computes the same stale `$yield`, then `noteDeposit` calls `_updateAssets(..., isDeposit=true)`, so `assetQueue.deposits += amount` queues that same yield again. When `submitQueuedAs` sets flushes it, the Hub treats it as another real holding increase, even though the actual `PoolEscrow` aToken balance only increased once. This increases NAV and consequently inflates share price if the NAV hook updates share price, since issuance remains the same.

$$\text{pricePoolPerShare} = \text{NAV} / \text{issuance}$$

The loss path also has a stale accounting issue because it calls the 5-arg `noteWithdraw(...)`, which internally calls `noteWithdraw` with `updateEscrow = false`, causing a similar `skip`.

Recommended Mitigation: Use `noteDeposit(..., updateEscrow=true)` and `noteWithdraw(..., updateEscrow=true)` in `aave-account`.

Client: Fixed in commit [ec5171a](#)

BurraSec: Resolved: the Aave account helpers now route yield/loss through `updateEscrow=true`, keeping `PoolEscrow.holding.total` in sync with rebasing `aToken` balances.

7.1.3 Share oracle workflow stores raw asset units as D18 price

Target: [update-share-oracle-price.json](#), [spoke-update-oracle-price.json](#), [OracleValuation.sol](#), [PricingLib.sol](#).

Severity: High

- Impact: High
- Likelihood: High

Description: The `update_share_oracle_price` template reads the share token decimals, computes one full share unit, and passes that value into `vault.convertToAssets(...)`. The returned `$vaultPrice` is a raw underlying asset amount denominated in the underlying asset's decimals, but the template forwards it directly as `price` to `spoke_update_oracle_price` which documents and encodes this value as a D18 scaled oracle price, and `OracleValuation.untrustedCall` stores the encoded value as `D18.wrap(newPrice)`. Downstream valuation through `Holdings.update` then treats the stored value as a D18 price and separately applies asset and pool decimals when quoting `holdings` through `PricingLib.convertWithPrice`. For a USDC-backed vault, `convertToAssets(10 ** shareDecimals)` returns approximately $1e6$ raw USDC units for a 1:1 share price, but the oracle price should be approximately $1e18$. As a result, generated account/claim workflows for non-18-decimal underlying assets can publish prices that are under-scaled by the underlying asset decimal factor, materially misvaluing share holdings and NAV/accounting that depend on the oracle price.

Recommended Mitigation: Normalize the `convertToAssets` return value from underlying asset decimals to D18 before calling `spoke_update_oracle_price`, for example:

```
priceD18 = convertToAssets(10 ** shareDecimals) * 1e18 / 10 ** assetDecimals
```

Client: Fixed in commit [a34e406](#).

BurraSec: Resolved: `update_share_oracle_price` now normalizes the raw `convertToAssets` result to D18 before publishing the oracle price.

7.1.4 ERC-7540 receipt token collision merges deposit and redeem accounting

Target: [accountingtoken-to-token-id.json](#), [erc7540-request-deposit.json](#), [erc7540-request-redeem.json](#), [erc7540-account-redeem.json](#), [erc7540-claim-deposit.json](#), [erc7540-claim-redeem.json](#), [AccountingToken.sol](#).

Severity: High

- Impact: Medium
- Likelihood: High

Description: The shared `accountingtoken_to_token_id` helper derives accounting-token ids from `(poolId, asset, isLiability)` and hardcodes `isLiability = false`. `AccountingToken.toTokenId` only encodes the pool id, asset address, and liability `bit`, so every workflow that passes the same `$asset` receives the same ERC-6909 id.

That id is used for two different receipt domains. In `erc7540_requestDeposit`, the id represents a pending deposit receipt and the minted amount is the requested underlying asset `amount`. In `erc7540_requestRedeem`, the workflow derives the same id from the same underlying `$asset`, but the minted `$amount` is the number of shares submitted to `requestRedeem`. The claim paths confirm the mismatch: `erc7540_claimDeposit` burns the receipt by `$assetsConsumed`, while `erc7540_claimRedeem` burns the same `$asset`-derived receipt by `$sharesClaimed`.

This makes the `BalanceSheet` asset identity ambiguous. A single accounting-token id can represent pending deposit assets, pending redeem shares, or a mixture of both. The redeem path is especially problematic because `erc7540_accountRedeem` prices that same accounting-token asset id as a pending share-redemption position using the vault share `price`, while the id itself is derived from the underlying asset. Since `AccountingToken.decimals(id)` are also inherited from the encoded asset address, redeem receipts for a 6-decimal underlying

can be registered and valued as if share-denominated raw amounts had 6 decimals. This can misstate in-flight holdings/NAV whenever redeem receipts exist, and the error compounds if deposit and redeem receipts for the same pool and underlying asset coexist under the same id.

Recommended Mitigation: Derive redeem receipt ids from the share token instead of the underlying asset. Concretely, update `erc7540_requestRedeem`, `erc7540_claimRedeem`, and `erc7540_accountRedeem` to call the existing `accountingtoken_to_token_id` helper with `$share` rather than `$asset`.

```
deposit receipt id = toTokenId(poolId, underlyingAsset, false)
redeem receipt id = toTokenId(poolId, shareToken, false)
```

This also aligns metadata and valuation with the represented unit, because redeem receipts inherit share-token decimals and can be priced using the share price, while deposit receipts continue to inherit underlying-asset decimals and represent pending asset amounts.

Client: Fixed in commit [37e6cb3](#)

BurraSec: Resolved: ERC-7540 redeem receipt IDs are now derived from the share token, while deposit receipt IDs remain derived from the underlying asset.

7.1.5 `cctp_claim` Strategist-Controlled Rawcalldata Enables Arbitrary `MessageTransmitter` Invocation

Target: [cctp-claim.json](#), [cctp.ts](#)

Severity:

- Impact: High
- Likelihood: Medium

Description: The `cctp*_claim` workflow template declares both `bytes` parameters of `receiveMessage(bytes, bytes)` with `input: []` and no `runtimeVariables`, making them fully strategist-controlled at execution time. The Centrifuge SDK treats `bytes` as a variable-length parameter and collapses all such inputs into a single `rawcalldata` slot encoded with `CALL | FLAG_RAW (0x21)`. This means the strategist supplies the entire calldata blob forwarded by `OnchainPM` to `MessageTransmitter`, including the 4-byte selector, and is not restricted to `receiveMessage`. On CCTP v2, entry-points such as `replaceDepositForBurn` and `replaceMessage` allow repointing the `mintRecipient` of an undelivered burn — a strategist who previously caused `OnchainPM` to initiate a burn can redirect those funds to an arbitrary address.

Recommended Mitigation: Remove the strategist-controlled `rawcalldata` pathway via a helper so the `rawcalldata` source is a computed return.

Client: Fixed in commit [9e5f85a](#) using SDK commit [46493d1](#).

BurraSec: Resolved: runtime CCTP message/attestation bytes are now encoded with the SDK's variable-length `0x80` path, keeping the `receiveMessage(bytes,bytes)` selector in the hashed command word so the strategist controls only Circle-validated arguments, not the called function selector.

7.1.6 ERC-7540 claim-redeem can permanently lock later fulfilled redemptions by over-burning accounting tokens

Target: [erc7540-claim-redeem.json](#), [AsyncRequestManager](#)

Severity:

- Impact: High
- Likelihood: Medium

Description: In `erc7540_claimRedeem`, the workflow stores the return value of `vault.withdraw(maxWithdraw, onchainPM, onchainPM)` as `$sharesClaimed` and burns that amount through `accounting_token_withdraw_and_burn`. This conflates the vault's rounded-up withdraw return with the exact accounting-token liability minted for the redemption. When the fulfilled asset/share ratio is not exactly representable in the D18 redeem price, the async redeem manager stores the redeem price with rounding down during fulfillment, while `withdraw()` converts assets

back to shares with rounding up. As a result, `$sharesClaimed` can exceed the fulfilled/requested share amount. Because accounting tokens are pooled by `(poolId, scId, accountingToken, tokenId)` rather than isolated per redemption, one valid claim can burn same-id accounting tokens backing another pending redeem. When that second redeem is later fulfilled, the approved claim-redeem workflow reverts in the burn path because the required liability has already been consumed. This can make valid fulfilled redemptions unclaimable through the canonical workflow until privileged/manual repair or a bespoke bypass is used. The issue is scoped to pooled same-id redeem accounting tokens; it does not require arbitrary-user or cross-pool burning.

Recommended Mitigation: Burn the exact fulfilled/requested redeem-share liability, not the rounded-up `withdraw()` return value. If using `maxRedeem`, first ensure it represents the exact claimable share liability and cap the burn to the accounting tokens minted for that redemption. Add a property test proving `erc7540_claimRedeem` never burns more accounting tokens than the fulfilled redemption share amount.

Client: Fixed in commit [2ff95b2](#)

BurraSec: Resolved: `erc7540_claimRedeem` now claims via `maxRedeem/redeem` and burns only the exact fulfilled share amount, avoiding over-burning from the rounded-up `withdraw()` return.

7.1.7 Oracle update target uses wrong bytes32 address encoding

Target: [spoke-update-oracle-price.json](#), [to-bytes32-address.json](#), [update-share-oracle-price.json](#)

Severity:

- Impact: Medium
- Likelihood: High

Description: `spoke_update_oracle_price` encodes the `Spoke.updateContract` target by calling `ScriptHelpers.toBytes32(address)` via `helpers_to_bytes32_address`. That helper puts the address in the **low 20 bytes** (`bytes32(uint256(uint160(addr)))`). Downstream, `MessageDispatcher.sendUntrustedContractUpdate` decodes the target with `CastLib.toAddress`, which requires the **low 12 bytes to be zero** and reads the address from the **high 20 bytes** (`bytes32(bytes20(addr))`). The conventions are opposite; the encoded target always reverts `PrefixNotZero()` at decode.

Once `ScriptHelpers` is deployed on mainnet, every workflow that reaches this step fails deterministically at `updateContract`. The template is composed into `update_share_oracle_price`, which is used by `erc4626_account`, `erc7540_accountRedeem`, and the claim paths `erc7540_claimDeposit` / `erc7540_claimRedeem` (via `update_share_oracle_price` at the end of each batch).

Recommended Mitigation: Do not use `helpers_to_bytes32_address` for `updateContract` targets. Have the generator compute `bytes32(bytes20(oracleValuation))` (address left-padded with 12 zero bytes) and pass it as a declared workflow variable. Keep `ScriptHelpers.toBytes32(address)` unchanged for CCTP and other consumers that expect low-padded encoding.

Client: Fixed in commit [4aa4ba0](#)

BurraSec: Fixed by pre-computing the `OracleValuation` contract address as a high-padded bytes32 at catalog build time.

7.2 Medium Risk

7.2.1 `cfg.scriptHelpers` Falsy Check Does Not Catch Zero Address String

Target: [workspaces/mainnet/generators/morpho.ts](#)

Severity:

- Impact: Medium
- Likelihood: High

Description: `if (!cfg.scriptHelpers)` `continue` guards loop workflow generation, but a zero-address string `"0x0000000000000000000000000000000000000000000000000000000000000000"` is truthy in JavaScript and passes the check. If an address file were to contain a zero address for `scriptHelpers`, the generator would emit loop workflows that call `mul()` against the zero address, causing every execution to revert.

Recommended Mitigation: Add an explicit zero-address check alongside the falsy check:

```
const ZERO_ADDRESS = "0x0000000000000000000000000000000000000000000000000000000000000000";
if (!cfg.scriptHelpers || cfg.scriptHelpers === ZERO_ADDRESS) continue;
```

This pattern should also be applied to other critical address guards in the generator (e.g. `aaveInfo.address`).

Client: The protocol accepts the risk and ensures that the workflow will not be run with a zero-address `scriptHelpers` value.

BurraSec: The protocol would still be vulnerable to a non-functional workflow if `scriptHelpers` is not properly configured. However, this issue is mitigated by validating that `scriptHelpers` is not the zero address before executing the scripts.

7.2.2 Share oracle workflow assumes underlying asset is 1:1 with pool currency

Target: [update-share-oracle-price.json](#), [spoke-update-oracle-price.json](#), [OracleValuation.sol](#), [IValuation.sol](#), [PricingLib.sol](#).

Severity: Medium

- Impact: Medium
- Likelihood: Medium

Description: The `update_share_oracle_price` template derives `$vaultPrice` from `vault.convertToAssets(10 ** shareDecimals)` and forwards it directly as the share asset's oracle `price`. After raw decimal normalization, this value represents `underlyingAssetPerShare`. However, `spoke_update_oracle_price` documents `price` as a D18-scaled `OracleValuation` price, and `OracleValuation` expects prices denominated in the pool currency, not in the vault's underlying asset. The valuation interface also defines asset valuations as pool-currency-denominated D18 prices.

This distinction matters because the protocol treats pool currency and vault underlying asset as separate price domains. Vault conversions use both `pricePoolPerShare` and `pricePoolPerAsset`, and tests explicitly cover cases where asset/share and pool/share differ, e.g. `priceAssetPerShare = pricePoolPerShare / pricePoolPerAsset`. Therefore, even if `$vaultPrice` is first normalized to D18, the template is only correct when the vault underlying asset is the pool currency or is valued 1:1 against it. For any non-1:1 underlying, the workflow publishes an asset /share value where pool/share is required, causing share holdings valued through `OracleValuation.getQuote` to be mispriced.

Recommended Mitigation: Convert the normalized asset-per-share value into pool currency before calling `spoke_update_oracle_price`:

```
assetPerShareD18 = convertToAssets(10 ** shareDecimals) * 1e18 / 10 ** assetDecimals;
poolPerShareD18 = assetPerShareD18 * pricePoolPerAssetD18 / 1e18;
```

Client: Fixed in commit [27730ef](#)

BurraSec: Resolved: `update_share_oracle_price` now converts asset-per-share into pool-per-share using `pricePoolPerAsset` before publishing the share oracle price.

7.2.3 Inconsistent bytes32 Mint Recipient Encoding Across CCTP Send Templates May Lead to Unrecoverable Minting

Target: [cctp-send-auto.json](#), [cctp-send.json](#)

Severity:

- Impact: Medium
- Likelihood: Low

Description: The two CCTP send templates encode `$onOffRamp` as a bytes32 mintRecipient using different mechanisms.

`cctp-send.json` routes `$onOffRamp` through the `helpers_to_bytes32_address` ScriptHelpers call and passes the result `$onOffRampBytes32` to `depositForBurn`:

```
{ "use": "helpers_to_bytes32_address", "map": { "scriptHelpers": "$scriptHelpers", "value": "$onOffRamp" },  
  ↪ "returns": "$onOffRampBytes32" }
```

`cctp-send-auto.json` passes `$onOffRamp` directly to the same parameter of `depositForBurnWithHook`:

```
{ "parameter": "bytes32", "label": "Mint recipient (OnOffRamp as bytes32)", "input": ["$onOffRamp"] }
```

SECURITY.md documents that `$onOffRamp` is SDK-padded to bytes32 (address in upper 20 bytes, lower 12 bytes zero), which is the same encoding `helpers_to_bytes32_address` produces - so the resulting on-chain value is identical today. The risk is forward-looking: the two templates establish divergent patterns. A developer forking `cctp-send-auto.json` may read the direct-pass as the correct general convention and omit the helper for an address-typed variable that the SDK does not pre-pad. A mintRecipient pointing to an incorrectly encoded bytes32 causes the USDC to be minted to an unintended address on the destination chain with no recovery path.

Recommended Mitigation: Standardise on one approach across all CCTP send templates. Apply `helpers_to_bytes32_address` consistently in `cctp-send-auto.json` (removing the dependency on the SDK padding assumption).

Client: <https://github.com/centrifuge/workflows/pull/75/changes/bfb2ff213223a723c471639918fe1e33a2f09c8c>

BurraSec: The protocol has added explicit documentation to ensure that inconsistent encoding does not affect the existing code.

7.2.4 CCTP automatic send always reverts

Target: [cctp-send-auto.json](#), [cctp.ts](#), [weiroll](#)

Severity:

Impact: Medium Likelihood: High

Description: The `cctp_send_auto` template passes `$hookData` directly into `Circle TokenMessenger.depositForBurnWithHook(..., bytes hookData)`, but `$hookData` is pinned as a single 32-byte literal. `Weiroll` treats bytes inputs as variable-length values and expects the state slot to contain the ABI tail, i.e. length + padded data. The generated call therefore appends the raw cctp-forward word after the offset without the required bytes length word.

As a result, the calldata reaching `depositForBurnWithHook` is malformed. The selector is correct and Ethereum `TokenMessengerV2` supports the function, but Solidity ABI decoding reverts before Circle's function logic runs. The manual `cctp_send` path succeeds because it uses only static parameters and does not pass dynamic hook data.

Recommended Mitigation: Encode `hookData` as a proper dynamic bytes value before the `Circle` call. For example, route it through `helpers_encode_bytes32` and pass the returned `$encodedHookData` into `depositForBurnWithHook`, while adding `scriptHelpers` to the auto workflow variables.

Client: Fixed in commit [6055e0d](#).

BurraSec: `$onOffRamp` is included properly into the `depositForBurnWithHook` call.

7.2.5 Shared \$scId Circuit-Breaker Key Mixes Egress Flow Limits Across Assets and Units

Target: [workspaces](#), [CircuitBreaker.sol](#), etc.

Severity:

- Impact: Medium
- Likelihood: Medium

Description: Circuit-breaker tally actions use an under-scoped key across multiple egress workflows. The generated templates pass only `$scId` as the `CircuitBreakerGuard.tally(bytes32 key, uint256 amount, uint256 max, uint256 window)` key, while the guard stores cumulative usage as `cumulative[msg.sender][key][window]`.

During workflow execution, `msg.sender` is the pool's `OnchainPM`, and there is one `OnchainPM` per pool. As a result, all guarded egress flows for the same share class share a single counter when they use the same configured window. This mixes unrelated flow types and units under one bucket, including USDC bridge amounts, share transfers, and issue/revoke share amounts.

This can cause both over blocking and ineffective limits. A strategist can consume the shared bucket through one approved egress flow and cause other same-share-class egress workflows to revert with `ExceedsCumulativeLimit` for the remainder of the window. Separately, caps configured for one unit can be tripped by another: for example, a USDC-denominated cap can be exhausted by share-denominated issuance or transfer amounts because both are tallied under `$scId`.

Recommended Mitigation: Use a flow-specific, unit specific tally key instead of raw `$scId`. For example, derive the key from `poolId`, `scId`, `asset/token`, `direction`, and `workflow type`. Also avoid configurable windows where a single logical limit is expected; pin or standardize the window so equivalent flows cannot split counters across different window values.

Client: Fixed in commit [ad983b7](#)

BurraSec: Resolved: circuit-breaker tally keys are now flow-scoped constants instead of raw `$scId`, preventing unrelated egress workflows from sharing the same usage bucket.

7.3 Low Risk

7.3.1 Sync VaultKind Silently Falls Through to Async Templates

Target: [workspaces/shared/centrifuge.ts](#)

Severity:

- Impact: Medium
- Likelihood: Low

Description: The vault dispatch logic checks if (vault.kind === "SyncDepositAsyncRedeem") and routes all other kinds into the else branch, which generates the full Async template set (erc7540_requestDeposit, erc7540_claimDeposit, erc7540_requestRedeem, erc7540_claimRedeem, erc7540_account). When Centrifuge adds a new vault kind (e.g. Sync), the generator silently emits workflows backed by incompatible templates. These workflows would be included in the Merkle policy and exposed in the marketplace with incorrect behavior and no build-time error.

Recommended Mitigation: Add an explicit else if branch for each known kind, and a final else that either throws or logs a skip with a clear message:

```
if (vault.kind === "SyncDepositAsyncRedeem") {  
  // ...  
} else if (vault.kind === "Async") {  
  // ...  
} else {  
  console.warn(` Skipping ${prefix}: unknown vault kind ${vault.kind}`);  
  continue;  
}
```

Client: <https://github.com/centrifuge/workflows/pull/70/changes/f120cacda7afdef84cd2635466188ec37c0320d2>

BurraSec: The code now correctly handles all available vault types, and if Sync vaults are introduced in the future, the workflow code flow will not break.

7.3.2 CCTP maxFee Is Hardcoded Across All Routes

Target: [workspaces/mainnet/generators/cctp.ts](#)

Severity:

- Impact: Low
- Likelihood: Low

Description: maxFee is hardcoded to "10000000" (10 USDC at 6 decimals) for every source -> destination pair regardless of chain. Circle's CCTP V2 fee tables define variable fees per route that can be updated. If the required fee exceeds the hardcoded cap, the depositForBurn call reverts and the workflow is unusable. If the cap is too high, the strategist may overpay on low-fee routes with no on-chain enforcement.

Recommended Mitigation: Make maxFee a configurable runtime variable set at policy creation time, or at minimum read it from per-chain address files so it can be updated via `pnpm run snapshot:write` without a code change. CCTP's recommendation - <https://developers.circle.com/cctp/concepts/fees#fee-tables:~:text=Do%20not%20hardcode,refunded%20during%20minting>.

Client: <https://github.com/centrifuge/workflows/pull/74/changes/95c62cae35a07403344a1138bf08afee48401f55>

BurraSec: Configurable max fee added according to CCTP's safety guidelines.

7.3.3 chainName Not Sanitized - Hyphens in Workflow IDs on Testnet Chains

Target: [workspaces/shared/centrifuge.ts](#), [workspaces/mainnet/generators/balance-sheet.ts](#)

Severity:

- Impact: Low
- Likelihood: High

Description: `sanitizeId()` is applied to `token.symbol` but not to `chainName` when constructing the workflow ID prefix. Chains whose `blockchain.name` contains hyphens (`base-sepolia`, `arbitrum-sepolia`) produce IDs like `cfg_base-sepolia_sym_deposit` and `balancesheet_base-sepolia_sym_issue`, violating the `protocol_chain_asset_action` underscore format expected by the marketplace and validator. The same pattern appears in the cross-chain transfer ID in `centrifuge.ts` (`cfg_${srcChainName}_${sym}_transfer_to_${sanitizeId(destChainName)}`) - only `destChainName` is sanitized, not `srcChainName`).

Recommended Mitigation: `chainName` usage in an ID construction with `sanitizeId()`:

```
const prefix = `cfg_${sanitizeId(chainName)}_${sym}`;
```

Apply consistently in `centrifuge.ts` and `balance-sheet.ts`.

Client: <https://github.com/centrifuge/workflows/pull/68/changes/2e885fefec227156f494228f2149cf2bb8835e46>

BurraSec: This fix introduces sanitization in the final step of workflow creation, preventing issues caused by unsanitized chain names.

7.3.4 scriptHelpers Passed to Balance-Sheet Workflows Whose Templates Don't Declare It

Target: [workspaces/mainnet/generators/balance-sheet.ts](#)

Severity:

- Impact: Low
- Likelihood: High

Description: The generator passes `scriptHelpers: cfg.scriptHelpers` in the `variables` object for all three balance-sheet workflows (`balancesheet_issue`, `balancesheet_revoke`, `balancesheet_transfer_shares_from`). None of these templates declare `scriptHelpers` in their `variables` array - they only declare `balanceSheet` and `circuitBreakerGuard`. The extra key is dead weight in every emitted workflow JSON and creates a mismatch between what the generator provides and what the templates actually need.

Recommended Mitigation: Remove `scriptHelpers` from the `variables` object in the balance-sheet generator. If the templates ever need script helper functionality in the future, add it to the template declaration at that point.

Client: <https://github.com/centrifuge/workflows/pull/67/changes/cc5ae98c8edb105f08b2b4b57d60eb6f99f60661>

BurraSec: The unused `scriptHelpers` variable has been removed.

7.3.5 Wrong Parameter Labels on requestDeposit and requestRedeem

Target: [workspaces/shared/templates/erc7540-request-deposit.json](#), [workspaces/shared/templates/erc7540-request-redeem.json](#)

Severity:

- Impact: Low
- Likelihood: High

Description: Both templates call a function (uint256,address,address) whose second and third parameters are controller and owner per the ERC-7540 spec (requestDeposit(uint256 assets, address controller, address owner) / requestRedeem(uint256 shares, address controller, address owner)). In both templates the second parameter is labeled "Owner" and the third is labeled "Receiver", inverting the names and omitting controller entirely. Because both inputs resolve to \$onchainPM (same address), the encoded calldata is correct and there is no runtime impact today. However, the wrong labels create confusion for anyone reading the templates, and would cause an incorrect parameter to be passed to the wrong slot if the controller or owner ever differ.

Recommended Mitigation: Update the labels in both templates to match the actual ABI parameter names:

- Second input label: "Controller" (currently "Owner")
- Third input label: "Owner" (currently "Receiver")

Client: <https://github.com/centrifuge/workflows/pull/63/changes/483ae3dd9d0a47c58a2a7f8cb09edbd7c6e81e9e>

BurraSec: The JSON schema parameter order now matches the smart contract parameter order.

7.3.6 Morpho loop callback can permanently strand excess repayment in FlashLoanHelper

Target: [morpho-loop-deposit.json](#), [morpho-loop-deposit-callback.json](#), [FlashLoanHelper.sol](#), [morpho.ts](#)

Severity:

- **Impact:** Medium
- **Likelihood:** Low

Description: The Morpho loop callback derives repayment from the OnchainPM's full token balance instead of the authoritative Aave flashloan amount. The outer workflow requests a computed \$flashAmount, and FlashLoanHelper receives the real Aave callback values, but only forwards the borrowed amount to the OnchainPM before executing the callback.

The callback then reads `asset.balanceOf($onchainPM)`, computes repayment using a pinned \$flashPremiumBps, borrows that amount from Morpho, and transfers it to FlashLoanHelper. When the OnchainPM has unexpected same-asset balance, the callback can transfer more than the real Aave debt. Aave only pulls `amount + premium`, leaving surplus stuck in FlashLoanHelper, which has no recovery path.

This is latent because Morpho loop workflows are currently disabled through `LEVERAGE_LEVELS = []`, and there is no obvious normal workflow path that pre-funds the OnchainPM with the same asset.

Recommended Mitigation: Pass the actual Aave amount and premium into the callback, derive repayment from `amount + premium`, reject unexpected same-asset OnchainPM balances, and add a restricted rescue path for accidental surplus in FlashLoanHelper.

Client: Fixed in commit [f93d8f1](#)

BurraSec: Resolved: the Morpho loop callback no longer derives repayment from the OnchainPM token balance and instead sizes repayment from the workflow's authoritative flash amount.

7.3.7 `erc7540_claimRedeem` Missing Slippage Guard

Target: [erc7540-claim-redeem.json](#)

Severity:

- Impact: Low
- Likelihood: Medium

Description: The `erc7540_claimRedeem` template has no `guards` block, so redemption settlements run with no slippage check, unlike `erc7540_claimDeposit`, which guards the analogous deposit-side exchange with `"slippageBps": 1000`.

Both key values produced by the settlement, `$assetsClaimable` (from `maxWithdraw`) and `$sharesClaimed` (returned by `vault.withdraw`), are strategist-influenced through the timing and sizing of the original `requestRedeem`. The threat model treats the strategist as adversarial. Without a guard, a strategist can execute `claimRedeem` when the fulfilled redemption rate is unfavorable, burning more accounting-token value than the assets deposited into the `BalanceSheet` are worth, and the workflow succeeds silently.

Recommended Mitigation: Add a slippage guard.

Client: Fixed in commit [c1f3342b355139a8b08929f372044bb8be7ac736](#)

BurraSec: Slippage guard was added as recommended.

7.3.8 Aave Templates Operate on Full `aToken` Balance Instead of the Exact Amount

Target: [aave-supply.json](#), [aave-withdraw.json](#)

Severity:

- Impact: Medium
- Likelihood: Low

Description: Both Aave workflow templates operate on OnchainPM's full `aToken` balance rather than the specific amount involved in the operation, allowing stray `aTokens` to inflate or deflate pool NAV.

In `aave-supply.json`, after supplying underlying to Aave the template reads `balanceOf(OnchainPM)` and passes the entire result to `bs_approve_and_deposit`. Any `aTokens` already held by OnchainPM, from a donation, a previous partial operation, or per-block interest rebase, get credited to `BalanceSheet` on top of the freshly minted `aTokens`. `SlippageGuard` only checks for losses, so over-credits pass through silently.

In `aave-withdraw.json`, the redemption amount is hard-coded to `uint256.max` (115792089237316195423570985008687907853269984665640564039457584007913129639935), causing Aave to redeem every `aToken` OnchainPM holds. If OnchainPM holds `aTokens` beyond the `BalanceSheet`-debited amount (e.g. from donations or a second position in the same asset), those are redeemed and deposited back asymmetrically, the `BalanceSheet` debit does not match the actual redemption.

Recommended Mitigation: In `aave-supply.json`, compute the deposit amount as the delta between post-supply and pre-supply `aToken` balances (`balanceAfter - balanceBefore`) rather than the full balance. In `aave-withdraw.json`, pass `$amount` (the `BalanceSheet`-debited `aToken` count) as the redemption argument instead of `uint256.max`.

Client: Fixed in commit [761dd7d](#)

BurraSec: Resolved: `aave_supply` now deposits only the post/pre `aToken` balance delta, and `aave_withdraw` now redeems exactly `$amount` instead of `uint256.max`, preventing stray OnchainPM `aTokens` from affecting `BalanceSheet`/NAV.

7.3.9 BalanceSheet Revoke Can Burn Any Holder's Shares Approved to OnchainPM

Target: [balancesheet-revoke.json](#), [BalanceSheet.sol](#), [ShareToken.sol](#), [ERC20.sol](#).

Severity:

- Impact: Medium
- Likelihood: Low

Description: The `balancesheet-revoke` workflow allows the executor to provide an arbitrary runtime `$user`, while the transfer step pins sender and to to `$onchainPM`.

`BalanceSheet.transferSharesFrom` then forwards those arguments directly to the share token via `token.authTransferFrom(sender_, from, to, amount)`. Although `authTransferFrom` is permissioned, it still calls the ERC20 transfer path with the supplied sender and from values. In `ERC20._transferFrom`, when `from != sender`, the transfer is authorized by `allowance[from][sender]`.

As a result, any holder who has approved the shared `OnchainPM` for the relevant share token can have their shares pulled by any strategist authorized to execute this workflow for the same pool/share class. The executor only needs to set `$user` to that holder and `$amount` within the holder's balance and allowance. The workflow then immediately calls `BalanceSheet.revoke`, burning the transferred shares from `OnchainPM`.

This makes approval to `OnchainPM` broader than the approving holder may expect. A user or strategist may approve `OnchainPM` so their own shares can be revoked, but that allowance is not bound to the workflow caller. It can also be reused by any authorized executor of the generic revoke workflow.

Recommended Mitigation: Pin the `from` address in the revoke workflow.

Client: If this particular workflow is enabled, the strategist is trusted to be able to burn any holder's shares.

BurraSec: Acknowledged.

7.3.10 Generator workflow IDs collide and silently drop vaults

Target: [centrifuge.ts](#)

Severity:

- Impact: Medium
- Likelihood: Low

Description: Centrifuge, Morpho, and Aave generators build workflow `id` values from chain name and sanitized token or vault symbol, not from vault address. When multiple vaults on one chain share a symbol, the generator emits several objects with the same `id` but different `variables.vault`. `generate-workflows` writes one file per `id` and overwrites on collision without warning. `validate` only errors on duplicate `id` when two files coexist under `out/`; after one generate pass there is at most one file per `id`, so it can pass while vaults were dropped in memory.

Vaults that lose the collision never appear in the published marketplace catalog. Strategists and integrators only see workflows for the surviving vault (last in generator loop order). Users of dropped vaults cannot select correct `variables.vault` from the catalog and may assume the product is unsupported. This does not corrupt on-chain state for the survivor vault; it omits tooling for the others. A maintainer updates `data/snapshots/centrifuge.json` with multiple vaults sharing a symbol on one chain (for example four Ethereum vaults with symbol `deJTRSY`). `pnpm run build:mainnet` runs the Centrifuge generator, then `combine` publishes `workflows.json` to IPFS. Only one `cfg_ethereum_dejtrsy_claim_deposit` file remains, the other three vaults have no generated workflows.

```
const sym = sanitizeId(token.symbol);
const prefix = `cfg_${chainName}_${sym}`;
id: `${prefix}_${suffix}`,
```

Recommended Mitigation: Include a stable unique key in every workflow `id` (for example `vault.id` or vault address). Track seen `ids` during generation and fail the build on collision.

Client: Fixed in commit [79abe34](#)

BurraSec: Generators now disambiguate workflow IDs with vault/market keys when symbols collide

7.3.11 Build and publish skip validate

Target: [publish-build.yml](#), [validate.ts](#)

Severity:

- Impact: Low
- Likelihood: Low

Description: build:mainnet and build run generate-workflows and combine only. validate is a separate script and is not chained into build scripts. publish-build.yml runs pnpm run build and uploads out/ to GitHub Pages and IPFS without running validate.

```
"build:mainnet": "tsx src/generate-workflows.ts mainnet && tsx src/combine.ts mainnet",  
"build": "pnpm run build:mainnet && pnpm run build:testnet && pnpm run generate:html",  
"validate": "tsx src/validate.ts"
```

validate runs in security-review only for PRs that touch template or generator paths. PRs that change only data/snapshots/, data/addresses/, src/combine.ts, or release wiring can merge with a green publish job and never run validate. Structural catalog defects that validate would catch (broken template references, missing required variables, malformed address-typed values, excessive slippageBps) can therefore reach production workflows.json on IPFS without a failed build. This removes a gate that would block obvious wiring errors before release. A maintainer merges a snapshot- or address-only change, publish-build succeeds, main pins workflow s.json to IPFS, and strategists load it via the SDK or WORKFLOWS_CID. Broken entries surface only when someone builds a policy or executes a workflow, not at catalog publish time.

Recommended Mitigation: Chain pnpm run validate after generate and after combine in build scripts with fail-closed behavior. Add validate to publish-build.yml before artifact upload and IPFS pin. Broaden CI so every catalog publish runs validate.

Client: Fixed in commit [28f598e](#)

BurraSec: Fixed by adding validate to to publish-build.yaml.

7.4 Informational

7.4.1 ERC-7540 workflows omit required queued-asset submissions

Target: [erc7540-request-deposit.json](#), [erc7540-request-redeem.json](#), [erc7540-claim-redeem.json](#), [bs-withdraw.json](#), [bs-approve-and-deposit.json](#), [accounting-token-mint-and-deposit.json](#), [BalanceSheet.sol](#), [QueueManager.sol](#).

Severity: Info

Description: The ERC-7540 workflows perform multi-asset BalanceSheet moves, but do not submit every affected queued asset before ending. In `erc7540_requestDeposit`, the workflow withdraws the source `$asset`, requests deposit and then mints and deposits the accounting token through `accounting_token_mint_and_deposit`, but finally submits only `$tokenAssetId`. The same pattern exists in `erc7540_requestRedeem`, where the workflow withdraws, deposits the accounting token, and again submits only `$tokenAssetId`. `erc7540_claimRedeem` on the other hand, deposits the redeemed underlying `$asset`, burns the accounting token, and then submits only `$tokenAssetId` and `$shareAssetId`, omitting the redeemed underlying asset queue. Since `BalanceSheet.withdraw` and `BalanceSheet.deposit` queue asset decreases/increases through `_updateAssets`, submitting only part of the touched asset set can leave Hub accounting temporarily partial. This is recoverable because `QueueManager.sync` can later submit the omitted queues, but the generated workflows themselves end before doing so.

Recommended Mitigation: Add `spoke_asset_to_id` lookups for any missing `$asset` / `$share` ids, then submit all affected queues before the workflow ends.

Client: Fixed in commit [6454ebb](#)

BurraSec: Resolved: ERC-7540 workflows now resolve and submit all BalanceSheet asset queues touched by each request/claim flow before ending.

7.4.2 Mainnet address pins set ScriptHelpers to zero

Target: [ethereum.json](#)

Severity: Info

Description: The Ethereum address file pins `ScriptHelpers` and `AccountingToken` to the zero address. Many workflow templates call helper actions that require `$scriptHelpers` for encoding and arithmetic, or `$accountingToken` for ERC-7540 accounting steps. When those variables resolve to `0x0`, the corresponding calls target a non-contract and the batch reverts before core protocol actions run.

This causes broad denial of service on mainnet for workflows that depend on helpers or accounting token actions, including ERC-7540 claim paths, CCTP templates that use `helpers_to_bytes32_address` or `helpers_min`, and oracle update flows that chain `helpers_pow10`. This is a deployment-readiness gap, not a defect inside `ScriptHelpers` itself. Pinned zero values:

```
{
  "managers": {
    "AccountingToken": "0x0000000000000000000000000000000000",
    "ScriptHelpers": "0x0000000000000000000000000000000000"
  }
}
```

Example helper dependency:

```
{
  "use": "helpers_pow10",
  "map": { "scriptHelpers": "$scriptHelpers", "decimals": "$shareDecimals" }
}
```

Recommended Mitigation: Complete the TODO and publish non-zero `ScriptHelpers` and `AccountingToken` addresses for mainnet. Also consider adding build-time validation that rejects workflows whose required manager pins resolve to address zero.

Client: Acknowledged. All guard addresses will be updated after onchain PM deployment.

BurraSec: Acknowledged.

7.4.3 Guarded workflows run unguarded with zero pins

Target: [ethereum.json](#), [combine.ts](#)

Severity: Info

Description: The workflow build injects guard actions when a template declares `guards.slippageBps` or `guards.allowance`. Injected guard actions are marked `optional: true`. On mainnet, `SlippageGuard` and `ApprovalGuard` are pinned to the zero address, so the executor skips those calls while still running deposit, withdraw, bridge, and transfer steps.

Operators and policy authors may believe catalog workflows are slippage or allowance protected because templates declare `guards` metadata and the combined workflow lists guard action names. In practice, no slippage envelope, allowance verification, or circuit-breaker enforcement runs on mainnet today. Unexpected vault pricing or lingering token approvals during settlement are not bounded by guards.

Example pins:

```
{
  "guards": {
    "SlippageGuard": "0x00000000000000000000000000000000",
    "ApprovalGuard": "0x00000000000000000000000000000000"
  }
}
```

Recommended Mitigation: Deploy and pin non-zero guard addresses per chain, add build-time validation that fails when `guards.*` is set but guard pins are zero, and document that guard protection only applies when pins are non-zero.

Client: Acknowledged. All guard addresses will be updated after onchain PM deployment.

BurraSec: Acknowledged.

7.4.4 Rounding labels omit Zero

Target: [scale-decimals.json](#), [mul-div.json](#), [pow10.json](#)

Severity: Info

Description: Helper templates `helpers_scale_decimals`, `helpers_mul_div`, and `helpers_pow10` label the rounding argument as only 0 = down and 1 = up. `ScriptHelpers` and `MathLib.Rounding` also define `Zero` as value 2. For unsigned integer math, `Zero` matches `Down` in practice because both floor the result, so there is no on-chain execution impact. Any workflow that composes these helpers displays incomplete labels in the catalog and policy UI; execution still passes 0, 1, or 2 at runtime or policy configuration as allowed by the contract.

```
{
  "parameter": "uint8",
  "label": "Rounding (0=down, 1=up)"
}
```

```
/// @param rounding 0 = Down (floor), 1 = Up (ceil), 2 = Zero (toward zero).
function mulDiv(uint256 a, uint256 b, uint256 c, MathLib.Rounding rounding) external pure returns (uint256);
```

Recommended Mitigation: Update helper labels to document 0 = Down, 1 = Up, and 2 = Zero.

Client: Fixed in commit [9913c58](#)

BurraSec: Documentation was added as recommended.

7.4.5 Generated workflows reference missing JSON Schema path

Target: [centrifuge.ts](#), [generate-workflows.ts](#), [combine.ts](#), [workflow.schema.json](#)

Severity: Info

Description: Generators emit `$schema` on each generated workflow pointing at `../../../../../schemas/workflow.schema.json`. The repo only defines `workflow.schema.json` under `workflows/src/schemas/`. There is no `workflows/schemas/` directory. Per-file outputs under `out/` resolve that relative path to a non-existent file.

```
{
  "$schema": "../../../../../schemas/workflow.schema.json",
  "id": "cfg_ethereum_dejaaa_claim_deposit",
  "template": "erc7540_claimDeposit"
}
```

The published catalog omits `$schema: combine` strips it before writing `workflows.json`. The SDK and fork tests never resolve `$schema` for strategist execution, so there is no impact on on-chain execution. IDE validation and any future CI that AJV-validates per-file `out/` artifacts against `$schema` may fail or silently skip validation. `validate.ts` does not load `workflow.schema.json`, so the wrong pointer does not fail the current CI gate. A maintainer or developer who opens generated files under `out/mainnet/workflows/` locally or on GitHub Pages will see tools and follow `$schema` to a missing path.

Recommended Mitigation: Point generator `$schema` at `../../../../../src/schemas/workflow.schema.json`, or publish schemas under `workflows/schemas/`.

Client: Fixed in commit [399ca0c](#)

BurraSec: Path was fixed as recommended and validation was added to check path.

7.4.6 ERC-7540 request workflows can freeze PoolEscrow availability by double-reserving requested tokens

Target: [erc7540-request-deposit.json](#), [erc7540-request-redeem.json](#), [PoolEscrow](#)

Severity: Info

Description: In `erc7540_requestDeposit` and `erc7540_requestRedeem`, the generated workflow withdraws the requested asset/share from `BalanceSheet` before calling the ERC-7540 vault request. `bs_withdraw` uses the full `BalanceSheet` withdrawal path and decrements `PoolEscrow.holding.total`. The vault request path then moves the same token amount back into escrow and `AsyncRequestManager` calls `balanceSheet.reserve(amount)`, which increments `holding.reserved` without restoring `total`. The same request therefore reduces availability twice: for initial holding T and request sum S , accounting becomes $total = T - S$, $reserved = S$, and $available = \max(T - 2S, 0)$. A request sum of $S \geq T / 2$ freezes all remaining availability, and $S > T / 2$ creates $reserved > total$, causing later same-asset/share `PoolEscrow.withdraw` paths to revert. This affects intended same-pool/same-token generated workflows and needs no external protocol failure, only an approved executor choosing runtime amounts. It can also block otherwise claimable same-asset redeem payout paths. The freeze lasts while the request remains pending; cancellation alone does not repair it until the cancel-claim/unreserve step, or privileged/manual repair, occurs.

Recommended Mitigation: Do not combine a full `BalanceSheet` withdrawal with the ERC-7540 request manager's reserve path for the same token amount. Call the vault request from custody state that does not decrement `PoolEscrow.holding.total`, or restore `PoolEscrow` holding accounting when the raw token returns before reserving it. Add a regression test asserting that requesting A reduces availability by exactly A , not $2A$.

Client: The intended workflow is cross-pool only, where a Pool A manager requests deposits/redeems into vaults of other pools and not back into Pool A itself.

BurraSec: Based on the clarification of the intended cross-pool-only usage, we accept this as a design assumption and treat the same-pool scenario as not applicable.